

# B.Tech Project (CP302)

**Title:** Perpetual voting method using pairwise comparison

**Group:** 15

**Supervised By:**  
Dr. Manish Kumar

**Submitted By:**  
Radha (2022CSB1108)  
Shaikh Asra Swaleh (2022CSB1121)



# Contents



**RECAP**



**$\Delta$ -Weighting Schemes**



**Setbacks and Output**



**MultiWinner Voting**



**Graph-Based Generalisation**



**Introduction to Temporal  
Settings**



**Terminologies**



**Decision Problem**



**When Voters Change and  
Committee Should(NOT)**



**Electing Successive  
Committees**



**Temporal Fairness**



**References**

# INTRODUCTION



 **Voting systems:** Voting systems are formal methods for aggregating individual preferences into a collective decision.

 **Plurality**(winner is the most first-place votes) and **Borda Count**(points based on rankings) are simple, but can give unfair results such as candidate preferred by most voters overall may lose.

 **Condorcet Method:** A candidate who beats every other candidate in pairwise comparison is called the Condorcet winner. The method fails when no such candidate exists (**cycle:**  $A > B, B > C, C > A \rightarrow$  **Condorcet paradox**).

 **Schulze Method:** A Condorcet-compliant algorithm that resolves cycles by computing the **strongest paths** between candidates and selecting the winner consistently.

 **Perpetual Method:** An extension for repeated elections. It ensures fairness across multiple cycles by carrying over voter representation, preventing dominance by a single group, and maintaining long-term proportionality.

# **Problem Statement & Objectives**

- **Our project** aims to implement a system that provides:
  - **Fair winner selection**(Condorcet-compliant via Schulze method, with margin-based tie-breaking; if margins are equal, the lexicographically smallest candidate is chosen).
  - **Resistance to paradoxes**(resolving cycles using strongest paths).
  - Implement the **Perpetual Voting Algorithm** with pairwise comparisons.

# Methodology

- **Input Collection** – Voters have preferences whether to approve or disapprove the candidates(**Approval Ballots**).
- **Pairwise Comparison Matrix** – Construct a matrix showing how many voters prefer candidate A over candidate B.
- **Strongest Path Calculation** – Use **Floyd–Warshall algorithm** to find the strongest path between candidates.
- **Winner Determination** – Candidate who is unbeaten in strongest path comparisons becomes the winner.
- **Perpetual Extension** – Results are updated across multiple elections to ensure historical fairness.



# Implementation Details

---

 Language: **Python**

 Core Algorithm:

- Updating **Satisfaction Weights** in each round.
- **Pairwise Matrix** Construction –  $O(n^2)$  comparisons.
- **Strongest Path** Calculation (**Floyd–Warshall**) –  $O(n^3)$ .
- Winner Determination – Schulze rules applied.
- Perpetual Extension – Maintains history across rounds.

 Weighting  $\Delta$ AB Scheme:

- $\Delta = 1 \rightarrow$  If A is in approval ballot but not B.
- $\Delta = 0.5 \rightarrow$  If A and B both are in approval ballot.
- $\Delta = 0 \rightarrow$  None of them are in approval ballot.

 Static vs Dynamic Handling:

- **Static**  $\rightarrow$  Approval Ballots remain fixed for all the rounds.
- **Dynamic**  $\rightarrow$  Approval Ballots changes every round (captures real-world variation).

# Datasets Used



## **Poland Election Dataset (Static Ballots)**

- Real-world dataset with approval preferences.
- Verified correctness of Schulze implementation.



## **Spotify Dataset (Static Ballots)**

- Adapted user playlist preferences into approval sets.
- Used as test case for candidate popularity analysis.



## **User Input Simulation (Dynamic Ballots)**

- Interactive input: voters could change preferences round-by-round.
- Demonstrated perpetual fairness across multiple cycles.



## **Synthetic Dataset Generation**

- Inspired by real-world elections and recommender data.
- Goal: create benchmark data to test dynamic approval ballots.

# Results & Observations



## Static Ballots:

- If one candidate appears in every approval ballot → always wins.
- Verified using Spotify dataset.



## Dynamic Ballots:

- Winners change with evolving voter approvals.
- Demonstrated robustness against dominance by single candidate.



## Example Output (from code):

- Pairwise matrix generated correctly.
- Strongest path matrix resolves cycles fairly.
- In case of cyclic tie, the one with a win of largest margin is declared winner. If tie persists, lexicographically smallest candidate is the winner.



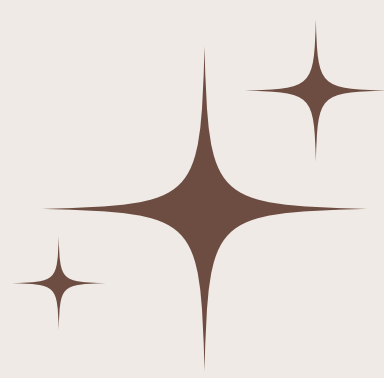
## Comparative Analysis:

- Schulze + Perpetual method provides long-term fairness.
- Outperforms plurality/Borda in avoiding paradoxes.



# Plan Ahead

---



## Explore $\Delta$ -weighting variations:

Investigate different ways of assigning weights to preferences to ensure greater fairness.



## Long-Term Goals:

- Extend to multi-winner elections
- Compare performance with other perpetual voting rules.

# $\Delta$ -Weighting Schemes



## 1st Formulation:

- $\Delta = 1 \rightarrow$  If A is in approval ballot but not B.
- $\Delta = 0.5 \rightarrow$  If A and B both are in approval ballot.
- $\Delta = 0 \rightarrow$  None of them are in approval ballot.



## 2nd Formulation: (History-Weighted Delta)

$$\Delta_{AB} = (\text{Score}(A) - \text{Score}(B)) + \lambda \cdot (\text{DrySpell}(A) - \text{DrySpell}(B))$$

- **Score(X)**: Current approval score for candidate X.
- **DrySpell(X)**: Number of consecutive rounds candidate X has not been selected.
- **$\lambda$** : Weighting parameter

# Axiomatic Analysis

---

 **Axioms:** Basic fairness principles used to evaluate voting rules.

 **Following are the three axioms:**

- **Simple Proportionality:** Yes, the new delta is sensitive to group sizes via approval scores
- **Independence of Unanimous Decisions:** Yes, unanimous approval for a candidate guarantees their score dominates.
- **Bounded Dry Spells:** Yes, the dry spell counter ensures persistent minorities eventually get represented.

# Outputs

## Classic Delta

### Output Of Poland Dataset 1:

```
00068-00001250.soi,12,10,12,12,10,12,12,12,10,12
```

### Output Of Poland Dataset 2:

```
00068-00000955.soi,8,8,8,11,8,8,8,11,8,8
```

### Output Of Poland Dataset 3:

```
00068-00001194.soi,7,7,7,7,7,7,7,7,7,7
```

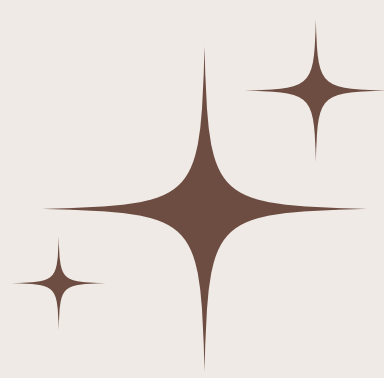
## History-Weighted Delta

```
File: 00068-00001250.soi: 12 10 12 10 12 2 5 10 12 13
```

```
File: 00068-00000955.soi: 8 8 8 11 4 8 9 5 8 11
```

```
File: 00068-00001194.soi: 7 7 2 7 1 7 2 6 7 1
```

# SetBacks



## **1. $\Delta$ stays constant for the whole round:**

It acts like a normalization factor, not true voter-level preference input.

## **2. Global approval scores are repeated for every voter:**

Score(A)–Score(B) gets multiplied again and again, causing artificial inflation.

## **3. Voter weight never changes the approval score itself:**

Weights only scales the final  $\Delta$ , so satisfaction influence becomes weak.

## **4. Dryspell dominates over approval in later rounds:**

The method moves toward diversity rather than optimal satisfaction.

## **5. Dryspell resets fully to zero after a win:**

The algorithm forgets past history, so long-term performance or contribution of a candidate is never stored or rewarded.



# Multiwinner Voting

Multiwinner voting is the process of selecting a group of winners (a committee) instead of a single candidate.

Some classical multiwinner voting rules:

- **AV(Approval Voting)**: Pick  $k$  candidates with the most approvals.
- **PAV(Proportional Approval Voting)**: Ensures proportional representation.
- **CCAV(Chamberlin–Courant)**: Each voter gets a representative.
- **SAV(Satisfaction Approval Voting)**: Maximize total satisfaction.
- **NAV/NSAV**: Approvals minus disapprovals

## Are Classical Multiwinner Voting Rules Enough?

Real committees often need structural constraints like—

Some candidates cannot work together

Some candidates should collaborate

Teams must stay connected and geographically close.

Committee must satisfy specific structural properties

# Graph-Based Generalisation

---

We construct a graph over candidates:  $G=(C,A)$

- **C**: set of candidates
- **A**: edges representing relations (conflict, cooperation, dependence, distance)

For a graph property  $P$  (e.g., “connected”, “independent”), a committee  $w$  is admissible iff:

$$w \in A \Leftrightarrow G[w] \in P$$

Where:

- $G[w]$  = the subgraph induced by the candidates in  $w$
- $P$  = a required structural property

**A committee  $w$  is valid only if it satisfies a graph-based property( $P$ ):**

- **Independent**: No conflicting edges
- **Connected**: Committee forms one cohesive group
- **Bounded radius**: Members are close to each other
- **Universal**: All subsets allowed (classical case)
- **Fixed-size**: Exactly  $k$  candidates (classical case)

# How Voting Rules Fit Into This Framework

Each multiwinner rule is expressed through a **scoring function**:

$$f : 2^C \times 2^C \rightarrow \mathbb{R}$$

**Score of a committee:**

$$\text{score}(w) = \sum_{v \in V} f(v, w)$$

Where,  $f(v, w)$  = "How happy is voter  $v$  with committee  $w$ ?"

Now the problem becomes:

$$\text{Find } w \in A_C^{\mathcal{G}} \text{ maximizing } \sum_{v \in V} f(v, w)$$

Among all committees allowed by the graph, pick the one with the highest total voter satisfaction

# Introduction to Temporal Settings

Classical multiwinner voting selects one committee for one election.

But in many real tasks, preferences change over time, and we need to elect a sequence of committees, one for each time step (day/event/year).

The classical model cannot ensure:

- **Stability:** committees shouldn't change too much
- **Adaptation:** committees sometimes should change a lot
- **Time consistency** across multiple stages

## Our Goal:

To choose committees  $C_1, C_2, \dots, C_\tau$  for each stage such that each committee:

- has enough support,
- has size at most  $k$ , and
- changes in a controlled way between consecutive stages.

This leads to two multistage models:

- **Conservative** (committees change little)
- **Revolutionary** (committees change a lot).

# Model for Temporal Setting

Find committees  $(C_1, C_2, \dots, C_\tau)$  that **satisfy**:

$$|C_t| \leq k, \quad \text{score}_t(C_t) \geq x$$

**Where Committee Score is :**

$$\text{score}_t(C_t) = |u_t^{-1}(C_t)|$$

**and either:**

$$|C_t \Delta C_{t+1}| \leq \ell$$

**OR**

$$|C_t \Delta C_{t+1}| \geq \ell$$

Conservative Model(MSNTV): Keep committees stable over time.

Revolutionary Model(RMSNTV): Force large changes between stages.



# Electing Successive Committees

Many real scenarios need a **sequence of committees** over time, with constraints on how often a candidate may serve.

Examples: weekly service teams, rotating duty groups, panel scheduling.

Classical multiwinner rules cannot:

- handle multiple rounds,
- limit how many times a candidate appears,
- enforce consecutive participation.

Complexity Dichotomy between computing series of committees of size

- **Size-2 committees** → mostly polynomial time
- **Size  $\geq 3$  committees** → mostly NP-hard

## Given:

- **Candidates:**  $C=\{c_1,\dots,c_m\}$
- **Voters:**  $V=\{v_1,\dots,v_n\}$
- **Number of committees:**  $t$
- **Committee size:**  $k$
- **Max frequency per candidate:**  $f$
- **Minimum quality required:**  $p$



# Terminologies

01

## COMMITTEE SERIES

A series of committees:  $S=(S_1, S_2, \dots, S_t)$   
Each  $S_i$  is a size- $k$  subset of candidates.

03

## EVALUATION FUNCTIONS

A function mapping a committee to a nonnegative integer is called a committee evaluation function.

- **App**: counts voters approving committee.
- **AppCC**: approval-based representation.
- **trCC**: 1 if top choice in committee.
- **eCC**: ranking-based extended representation.
- **CC**: classical representation via best-ranked member.
- **FWS**: sum of independent candidate scores.

02

## FREQUENCY CONSTRAINT

Each candidate may appear in at most  $f$  committees, and only in consecutive ones:

$$P(c) \leq f, P(c) = \{i \mid c \in S_i\}$$

and

$$\max P(c) - \min P(c) + 1 = |P(c)|$$

04

## COMMITTEE QUALITY FUNCTIONS

The output of evaluation function for a particular committee is called a committee quality

- Utilitarian quality

$$\text{util}(Y(S)) = \sum_{i=1}^t Y(S_i)$$

- Egalitarian quality

$$\text{egal}(Y(S)) = \min_{i \in [t]} Y(S_i)$$

## Decision Problem

### X-Y-Successive Committees Election (X-Y-SCE):

$$X \in \{util, egal\}, Y \in \{App, AppCC, trCC, eCC, CC\} \cup FWS$$

X = how we evaluate the entire sequence

Y = the scoring rule used to evaluate each committee

#### Ordinal Based

01

**CC: Chamberlin-Courant (Borda scores)**

$$CC(S) = \sum_{i \in [n]} \max_{j \in [k]} (m - \text{pos}_i(w_j))$$

02

**eCC: egalitarian Chamberlin-Courant:**

$$eCC(S) = \min_{i \in [n]} \max_{j \in [k]} (m - \text{pos}_i(w_j))$$

03

**F<sub>WS</sub>: A committee evaluation function Q, is weakly separable if, for some function**

$$\phi: [m] \rightarrow \mathbb{N}_0, Q(S) = \sum_{i \in [n]} \sum_{j \in [k]} \phi(\text{pos}_i(w_j)).$$

#### Approval Based

01

**App: Approval voting**

$$App(S) = \sum_{i \in [n]} |A(i) \cap S|,$$

02

**trCC<sup>α</sup>: threshold-α Chamberlin-Courant:**

$$trCC^\alpha(S) = \begin{cases} 1 & \text{if } |\{i \in [n] \mid A(i) \cap S \neq \emptyset\}| \geq \alpha n, \\ 0 & \text{otherwise.} \end{cases}$$

03

**AppCC: Chamberlin-Courant/MaxCover:**

$$AppCC(S) = |\{i \in [n] \mid A(i) \cap S \neq \emptyset\}|.$$

# Temporal Fairness

Fairness must consider:

- Whether a voter/group is represented eventually (global)
- Represented each round (local)
- Represented every few rounds (frequency-based)

## Main Mathematical Notions

They define general fairness conditions over time:

### 1. Global fairness:

Maximize total utility over time:

$$\max \sum_{r=1}^{\ell} \sum_{i \in N} u_{i,r}(o_r)$$

### 2. Local fairness:

Fairness must hold for every  $r$ (time).

$$\text{Fair}(o_r) = \text{true for all } r$$

### 3. Frequency fairness:

No voter should go more than  $k$  rounds without satisfaction:

$$\forall i : \quad \text{gap}(i) \leq \kappa$$

# References

---

☞ Schulze Method: [Wikipedia](#)

☞ Perpetual Voting Method:

- [Optimal Bounds for Dissatisfaction in Perpetual Voting](#)
- [Perpetual Voting: The Axiomatic Lens](#)
- [Justified Representation for Perpetual Voting](#)
- [Artificial Delegates Resolve Fairness Issues with Partial Turnout](#)
- [Perpetual Voting: Fairness in Long-Term Decision Making](#)
- [Proportional Aggregation of Preferences for Sequential Decision Making](#)
- [Proportional Decisions in Perpetual Voting](#)

☞ Other Research Papers:

- [Trends in Computational Social Choice](#)
- [Electing Successive Committees: Complexity and Algorithms](#)
- [Efficient Algorithms for Electing Successive Committees](#)
- [Multiwinner Voting with Restricted Admissible Sets: Complexity and Strategyproofness](#)
- [Temporal Fairness in Multiwinner Voting](#)
- [When Votes Change and Committees Should \(Not\)](#)
- [Strengthening Proportionality in Temporal Voting](#)

# Q & A